

Protegrity Arrow Code Map

In this document we summarize the changes made to the Arrow codebase that implement the PME extensions project.

The changes are limited to the python and cpp directories in the [main Protegrity Arrow git repository](#).

The [PR where the code was merged into the main branch](#) shows the changes made to each file.

Please note: These files have undergone some changes after the main merge PR. We have linked to the merge PR for simplicity, but please check the latest version in the Protegrity Arrow repository before doing any new merge operations.

This is only a very brief summary of the changes made in each file, and is intended as a guide only. Full details and rationale for the changes can be found in the full design documentation.

Python Changes

[python/pyarrow/includes/libarrow_dataset_parquet.pxd](#)

[python/pyarrow/includes/libparquet.pxd](#)

[python/pyarrow/includes/libparquet_encryption.pxd](#)

[python/pyarrow/parquet/encryption.py](#)

[python/pyarrow/_parquet.pxd](#)

[python/pyarrow/_parquet_encryption.pxd](#)

- Add definitions for the ColumnEncryptionAttributes, the ExternalFileEncryptionProperties, and the ExternalFileDecryptionProperties.

[python/pyarrow/src/arrow/python/parquet_encryption.cc](#)

[python/pyarrow/src/arrow/python/parquet_encryption.h](#)

- Add the CryptoFactory methods for getting ExternalFileEncryptionProperties and ExternalFileDecryptionProperties.

[python/pyarrow/_dataset_parquet_encryption.pyx](#)

[python/pyarrow/_parquet.pyx](#)

- Check encryption and decryption properties and cast them to the External versions if needed.

[python/pyarrow/_parquet_encryption.pyx](#)

- File with the bulk of the code for creating and ColumnEncryptionAttributes, ExternalFileEncryptionProperties, and ExternalFileDecryptionProperties. Contains constructors, getters and setters for all its properties and converts data structures from Python to their C++ equivalents.

[python/pyarrow/tests/test_dataset_encryption.py](#)

[python/pyarrow/tests/parquet/test_external_encryption.py](#)

- Round trip external encryption test.

[python/scripts/base_app.py](#)

[python/scripts/test_connection_config_file.json](#)

[python/scripts/use_external_dbpa_encryption.py](#)

[python/scripts/verify_external_roundtrip_correctness.py](#)

- Scripts for testing round trip external encryption, and for learning how to use the feature in apps.

C++ Changes

[cpp/src/arrow/dataset/CMakeLists.txt](#)

[cpp/src/arrow/dataset/file_parquet.cc](#)

[cpp/src/arrow/dataset/file_parquet_encryption_test.cc](#)

[cpp/src/arrow/dataset/parquet_encryption_config.h](#)

- Rename encryption_internal to encryption_utils.
- Add support in MakeReaderProperties and in MakeWriter for ExternalEncryption and ExternalDecryption Config objects.
- Add tests for External Encryption and Decryption config support.
- Extend ParquetEncryptionConfig and ParquetDecryptionConfig to support ExternalEncryption and ExternalDecryption Config objects.

[cpp/src/parquet/page_index.cc](#)

[cpp/src/parquet/encryption/file_key_wrapper.cc](#)

[cpp/src/parquet/encryption/key_toolkit_internal.cc](#)

- Rename encryption_internal to encryption_utils.

[cpp/src/arrow/util/io_util.cc](#)

[cpp/src/arrow/util/io_util.h](#)

- Added methods for closing a dynamic library

[cpp/src/generated/parquet_types.cpp](#)

[cpp/src/generated/parquet_types.h](#)

[cpp/src/generated/parquet_types.tcc](#)

- These files were not directly modified, but changed because of parquet.thrift and thrift_internal.h and the subsequent process to regenerate these files. See [this section](#) for reference.

[cpp/src/parquet/CMakeLists.txt](#)

[cpp/src/parquet/encryption/CMakeLists.txt](#)

- Change source names after the major encryptor and decryptor refactor.
- Add an external project for the DBPS agent code, for building and linking it, and for bringing its new dependencies as well.

[cpp/src/parquet/meson.build](#)

[cpp/subprojects/packagefiles/dbps_agent/meson.build](#)

[cpp/subprojects/packagefiles/magic-enum/meson.build](#)

[cpp/subprojects/packagefiles/tcb-span/meson.build](#)

[cpp/subprojects/dbps_agent.wrap](#)

[cpp/subprojects/magic-enum.wrap](#)

[cpp/subprojects/tcb-span.wrap](#)

- Change source names in meson.build file after the major encryptor and decryptor refactor.
- Add an external project for the DBPS agent code, for building and linking it, and for bringing its new dependencies as well.

[cpp/src/parquet/column_reader.cc](#)

[cpp/src/parquet/column_reader.h](#)

[cpp/src/parquet/encryption/encoding_properties.cc](#)

[cpp/src/parquet/encryption/encoding_properties.h](#)

[cpp/src/parquet/encryption/encoding_properties_test.cc](#)

[cpp/src/parquet/encryption/per_column_encryption_test.cc](#)

- Add a new EncodingProperties class that aggregates metadata related to encodings, and its test.

- Add a ColumnDescriptor object to the CryptoContext to store encoding properties.
- Add methods to build the EncodingProperties of a page by reading metadata from page headers and the ColumnDescriptor.
- Main change to the decrypt call! Get page encoding properties and update them before deciding which type of decrypt call to make: whether the decryptor can calculate the length of plaintext, or if it should decrypt with a managed memory buffer.

[cpp/src/parquet/column_writer_test.cc](#)

[cpp/src/parquet/column_writer.cc](#)

[cpp/src/parquet/file_deserialize_test.cc](#)

- Main change to the encrypt call! Get page encoding properties and update them before deciding which type of encrypt call to make: whether the encryptor can calculate the length of the ciphertext, or if it should encrypt with a managed memory buffer.
- Method that updates data encryptor metadata, and manages conflicts in the metadata keys. These metadata values will be written out to the Parquet file metadata directly.
- Adding encryption and metadata tests to the corresponding column writer test.
- Adding tests to file deserialization that check whether encodings are captured properly.

[cpp/src/parquet/file_reader.cc](#)

[cpp/src/parquet/file_writer.cc](#)

- Rename encryption_internal to encryption_utils.
- Forward new parameters needed in GetColumnDataDecryptorFactory and CryptoContext.
- Forward new parameters needed in GetColumnDataEncryptor.

[cpp/src/metadata.cc](#)

[cpp/src/metadata.h](#)

- Add the EncryptionAlgorithm to the ColumnCryptoMetaData object. Use Thrift changes to properly serialize the values of the new encryption algorithm enum. This is an important change, as it is where we write out to the metadata the column specific encryption algorithm.
- Expose fields needed during encryption and decryption to extract metadata related to the column (ColumnDescriptor, ReaderProperties, and WriterProperties).

- New method for adding key value metadata.
- Change encrypt metadata and encrypt footer calls to check whether the encryptor can calculate the ciphertext length or if it needs to call the managed buffer version of encrypt.
- **Important!** Fix a bug in Arrow where a plaintext footer would always be signed using AES_GCM_V1, and not the actual encryption algorithm used for the file. (this is unrelated to external encryption).

[cpp/src/parquet/parquet.thrift](#)
[cpp/src/parquet/thrift_internal.h](#)

- Add Thrift structures for ExternalDBPAV1 and extend the EncryptionAlgorithm union to include it.
- Add methods for converting to and from the Thrift version of the new enum value to the C++ version of the enum value.
- Explicitly add a method for ExternalDBPAV1 that sets AAD support to false.
- Modify Thrift message encryptor and decryptor calls to check whether they can calculate the lengths of the plaintext or ciphertext to know which version to call (using managed buffer or not).

[cpp/src/parquet/types.cc](#)
[cpp/src/parquet/types.h](#)
[cpp/src/parquet/types_test.cc](#)

- Add the new ExternalDBPAV1 enum value to the C++ ParquetCipher struct.
- Add a util method that checks whether a particular encryption algorithm is supported or not.
- Test for these changes.

[cpp/src/parquet/encryption/crypto_factory_test.cc](#)
[cpp/src/parquet/encryption/crypto_factory.cc](#)
[cpp/src/parquet/encryption/crypto_factory.h](#)
[cpp/src/parquet/encryption/encryption.cc](#)
[cpp/src/parquet/encryption/encryption.h](#)
[cpp/src/parquet/encryption/properties_test.cc](#)

- Add ParquetCipher to the ColumnEncryptionProperties.

- Create the ExternalFileDecryptionProperties and ExternalFileEncryptionProperties as subclasses of the FileDecryptionProperties and FileEncryptionProperties respectively. This includes Builder classes as well.
- Create the ExternalDecryptionConfiguration and ExternalEncryptionConfiguration as subclasses of the DecryptionConfiguration and EncryptionConfiguration respectively.
- Add methods that take the low level configurations and convert them to the higher level file properties in the CryptoFactory.
- Refactor the CryptoFactory code to extract common functions used by both types of file properties constructors.
- Perform basic validation in the CryptoFactory of the configurations sent in from applications.
- Add a test for the CryptoFactory, which did not exist previously, and modify the properties test to make sure all combinations, per-column values, and validations are set correctly.

[cpp/src/parquet/encryption/aes_encryption.cc \(was encryption_internal.cc\)](#)

[cpp/src/parquet/encryption/aes_encryption.h \(was encryption_internal.h\)](#)

[cpp/src/parquet/encryption/aes_encryption_nossl.cc \(was encryption_internal_nossl.cc\)](#)

[cpp/src/parquet/encryption/aes_encryption_test.cc \(was encryption_internal_test.cc\)](#)

[cpp/src/parquet/encryption/decryptor_interface.h](#)

[cpp/src/parquet/encryption/encryption_utils.cc](#)

[cpp/src/parquet/encryption/encryption_utils.h](#)

[cpp/src/parquet/encryption/encryptor_interface.h](#)

- Major encryptor and decryptor refactor in Arrow!
- See the design document for details on the refactor design.
- Defined new interfaces for encryptor and decryptor, independent of the encryption algorithm used.
- Renamed (and restructured) the previous default encryptor and decryptor to better reflect the algorithm used.
- Moved helper methods out to a utils class.

- Updated and added methods to conform to the new encryptor and decryptor interfaces.
- Adding tests for new methods and restructuring existing ones to new interfaces.

[cpp/src/parquet/encryption/internal_file_decryptor.cc](#)
[cpp/src/parquet/encryption/internal_file_decryptor.h](#)
[cpp/src/parquet/encryption/internal_file_encryptor.cc](#)
[cpp/src/parquet/encryption/internal_file_encryptor.h](#)

- Remove all hard-wire references to Aes encryptors and decryptors, and replace them with the new interfaces from the refactor.
- Add new methods to support the new interfaces for updating encoding properties, using managed buffers, and retrieving the key value metadata that will be written out to the Parquet file.
- Extend the method that creates the column encryptor and decryptor so that it receives a ColumnChunkMetaData (or its Builder) so that we can extract column metadata to send to the encryptor and decryptor.
- Change how the encryptors and decryptors are created and stored, using factories instead of fixed length arrays.
- Main change to encryptor and decryptor creation! Check whether the column specifies a column-specific encryption algorithm, and create the right type of encryptor and decryptor based on its value.

[cpp/src/parquet/encryption/external_dbpa_encryption_test.cc](#)
[cpp/src/parquet/encryption/external_dbpa_encryption_utils.h](#)
[cpp/src/parquet/encryption/external_dbpa_encryption.cc](#)
[cpp/src/parquet/encryption/external_dbpa_encryption.h](#)

- Implementation of the new encryptor and decryptor interface defined in the refactor for the new ExternalDBPAV1 algorithm.
- Logging utils and test suite that ensures round trip encryption and decryption works correctly.

[cpp/src/parquet/encryption/external/*](#)

- New code responsible for all functionality related to the DBPA encryptor and decryptor.

- Loads the library for the agents, receives the encrypt and decrypt calls (and other helper calls), performs verification, prepares the payload, and forwards the call to the library.
- Returns results to Arrow transparently.
- See project design documents for full details on interfaces and inner workings.

GitHub Workflow Changes

[.github/workflows/cpp.yml](#)
[.github/workflows/cpp_extra.yml](#)
[.github/workflows/docs.yml](#)
[.github/workflows/python.yml](#)
[.github/workflows/ruby.yml](#)
[ci/scripts/python_build.bat](#)

- Fixing several failing GitHub Actions, which were missing configurations for running external DBPA builds or tests.
- Some actions were fixed by adding values for LD_LIBRARY_PATH and DBPA_LIBRARY_PATH.
- Others had more complicated environments due to running outside of Docker with unknown setups. Explicitly turn off DBPS library building in these cases.
- This does not affect test coverage as the same tests pass in other environments/operating systems combinations.

<>